MINOR PROJECT ASSESSMENT

Vulnerability Assessment, API Testing & Forensics

Submitted by: Akula Harshini

Marking Scheme Reference

Section	Coverage	Marks
A	Web Vulnerabilities — SQLi + XSS	30
B	API Security Testing	40
C	Digital Forensics	30
	TOTAL	100

Q1. SQL Injection — bWAPP (15 Marks)

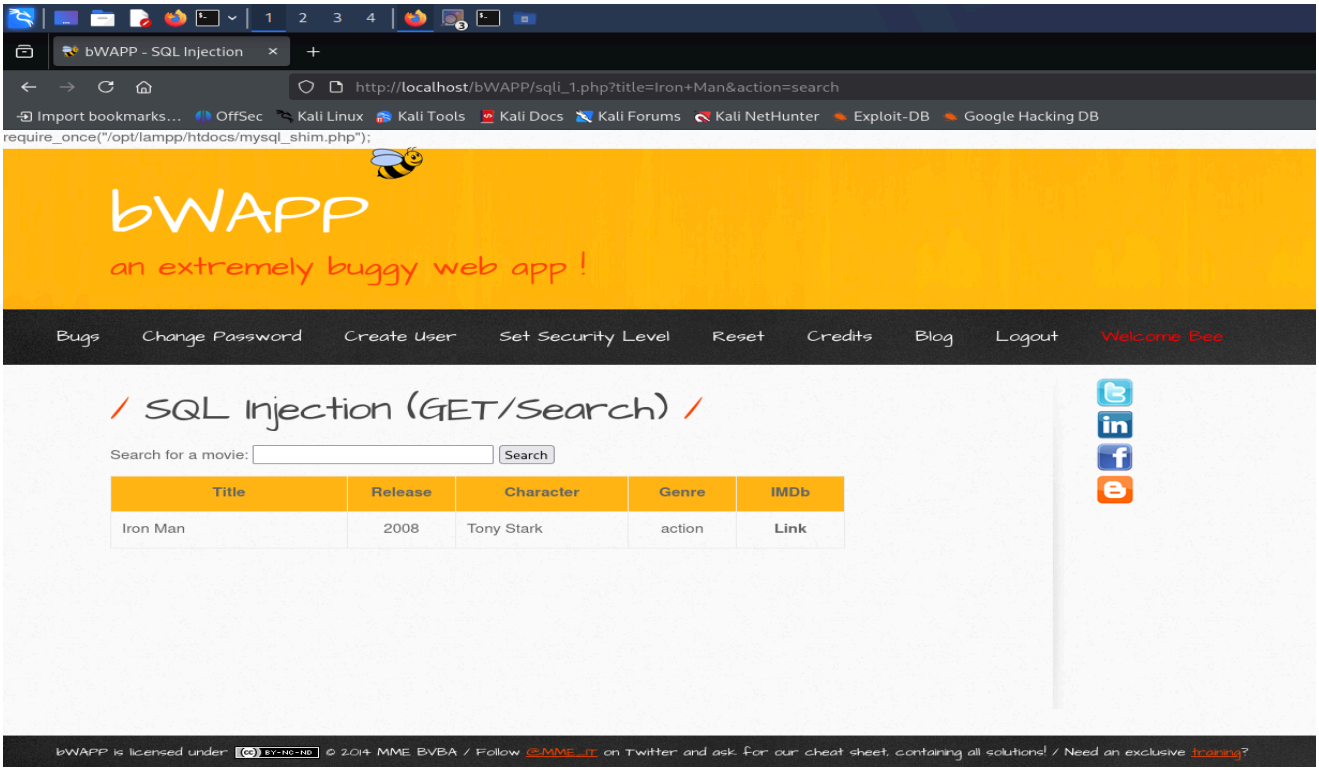
Objective

Identify an input point vulnerable to SQL Injection in bWAPP, establish a baseline, perform controlled SQLi testing, and document impact and remediation.

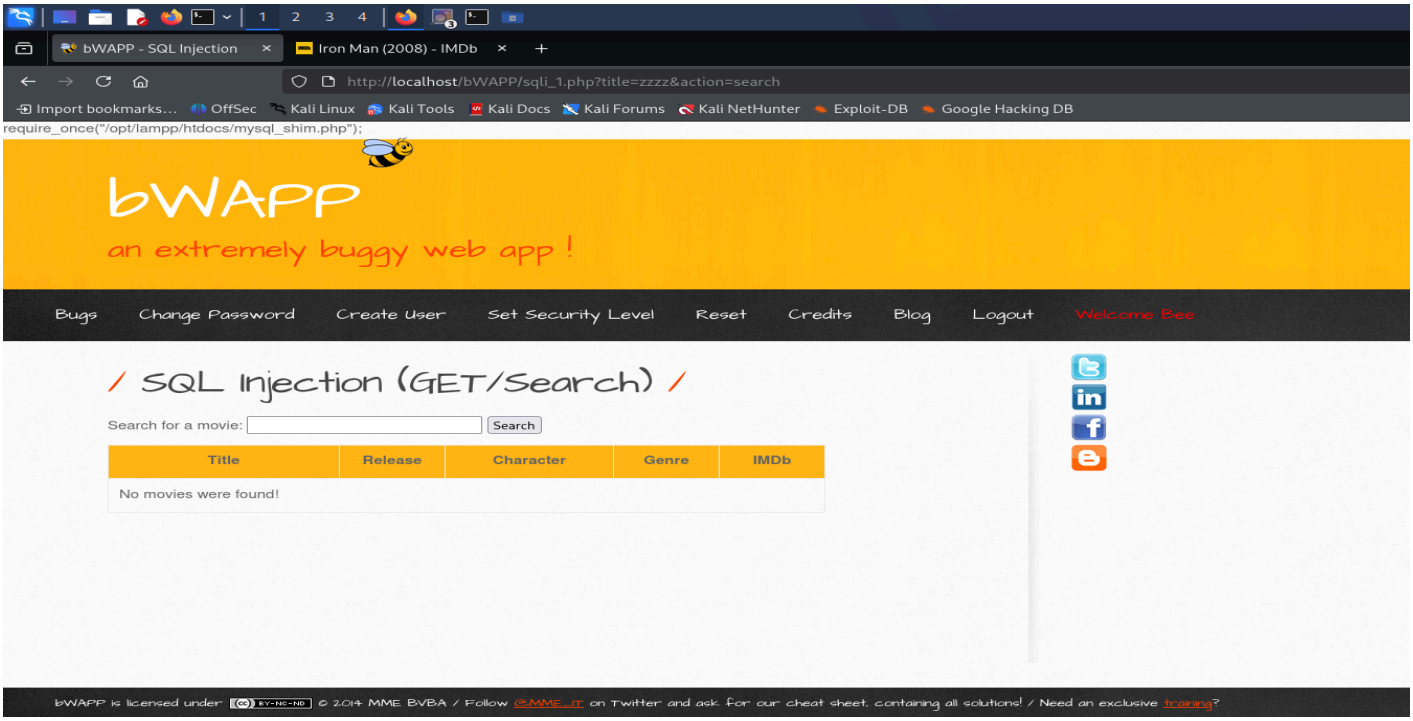
Methodology

1. Logged into bWAPP (security level: low) and selected the "SQL Injection (GET/Search)" module.
2. Established a baseline with a valid search term: `Iron Man`
3. Established a baseline with an invalid search term: `zzzz`
4. Tested a single quote `'` to check for unescaped input handling.
5. Tested a boolean-false payload: `' AND '1'='2'#`
6. Tested a boolean-true payload: `' AND '1'='1'#`

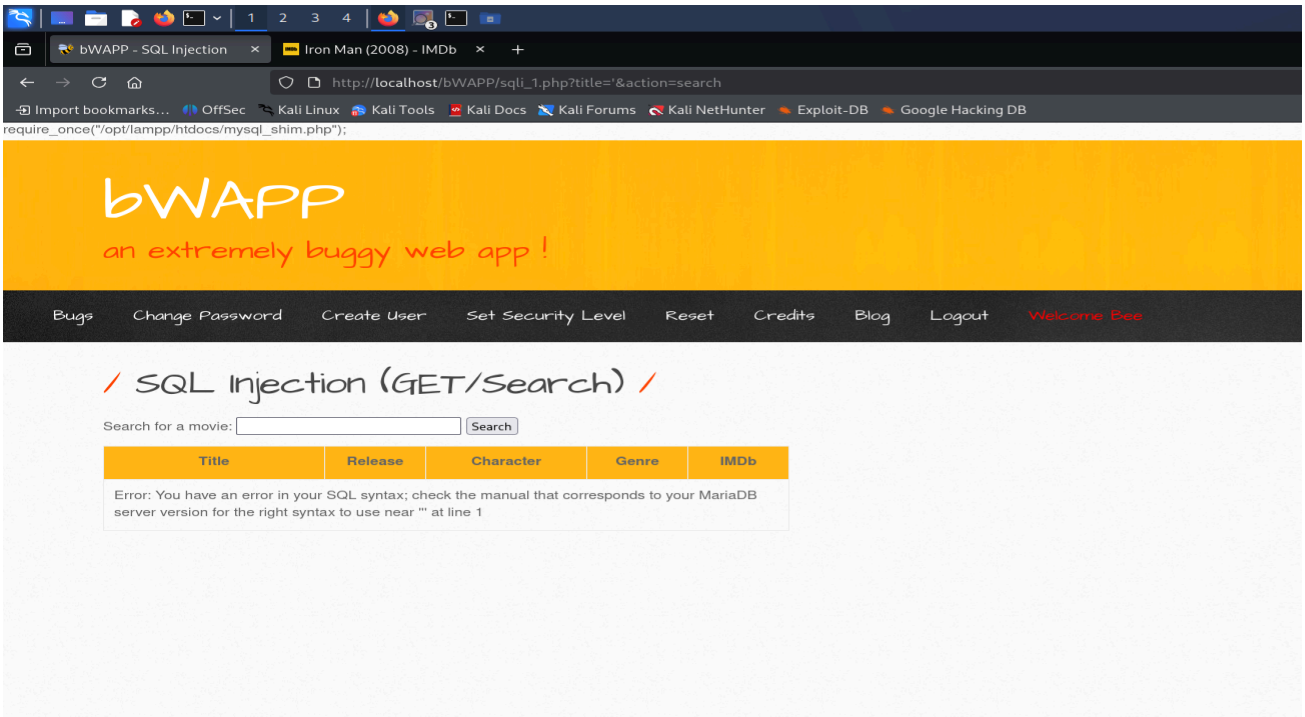
Evidence (Screenshots)



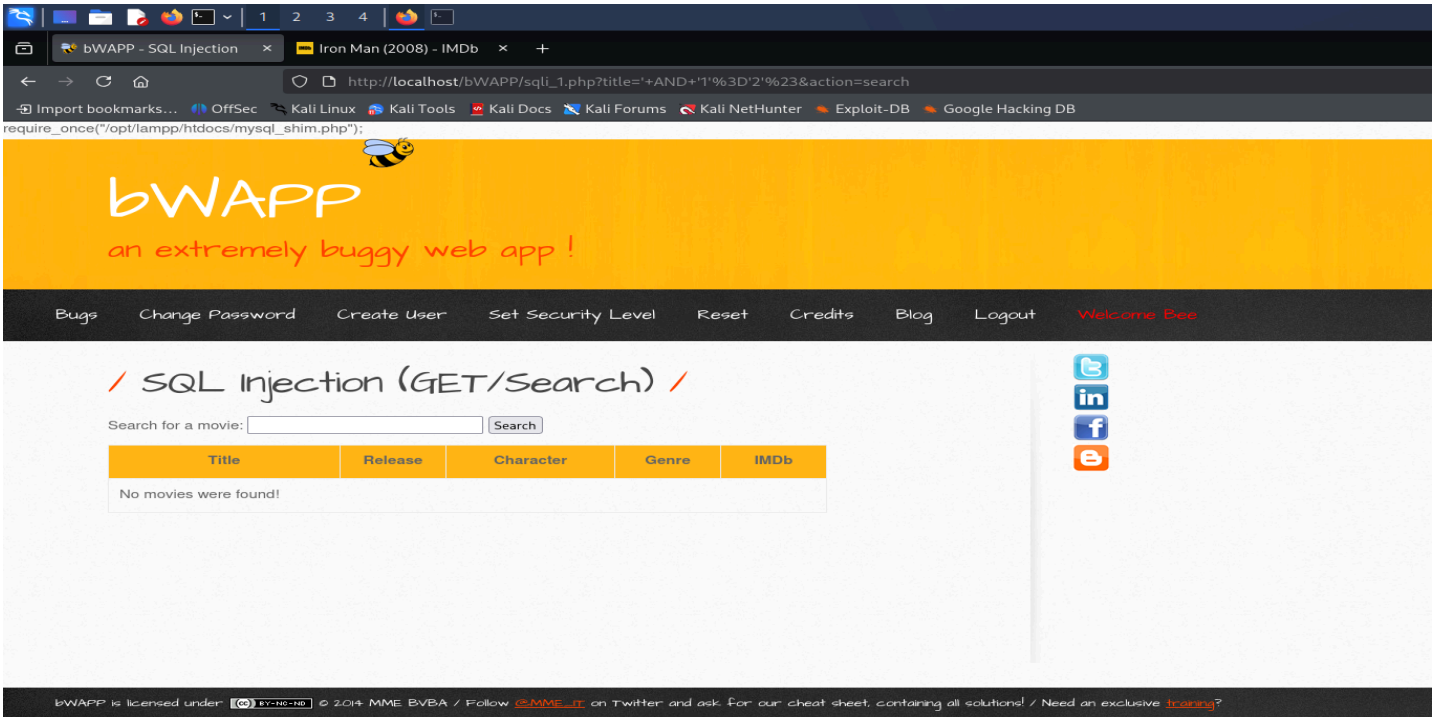
Screenshot 1: Baseline valid input (`Iron Man`) — returns 1 matching row



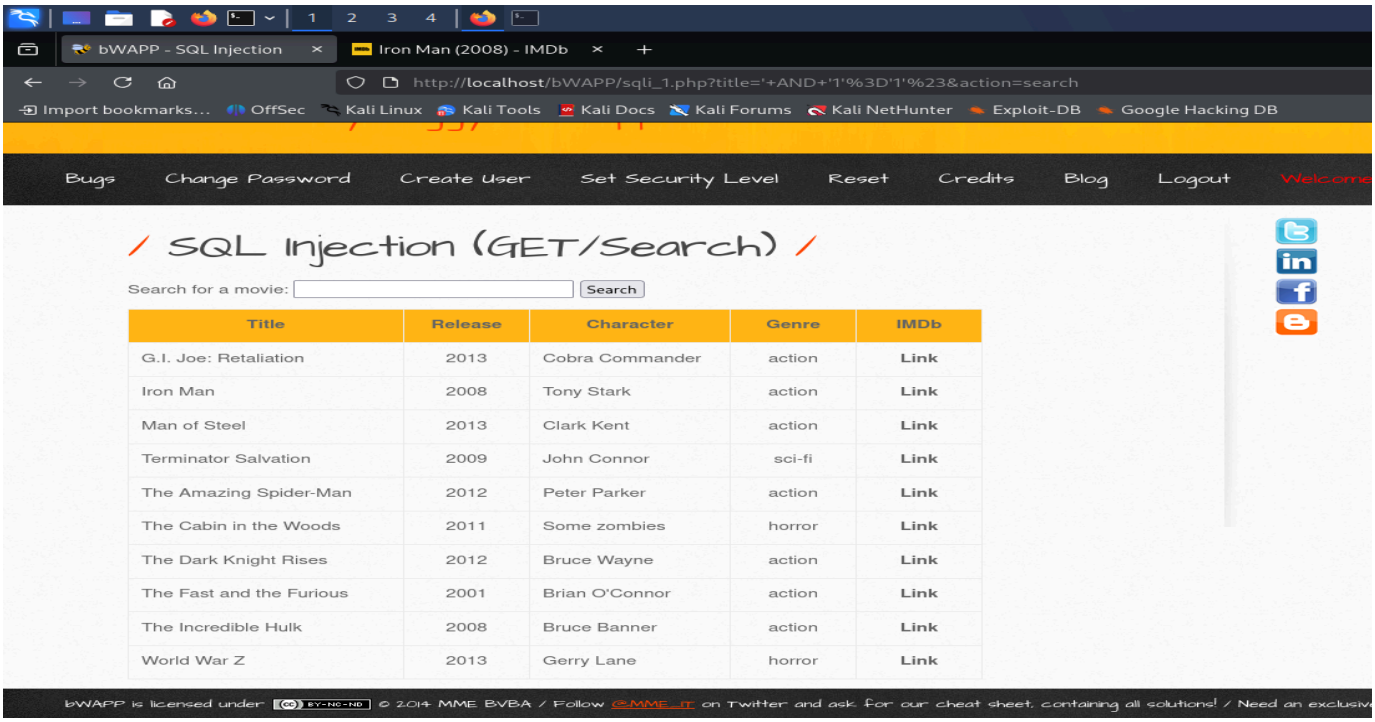
Screenshot 2: Baseline invalid input (zzzz) — "No movies were found!"



Screenshot 3: Single quote (') — MariaDB syntax error returned, quoting the broken input



Screenshot 4: Boolean false (`' AND '1'='2'#`) — "No movies were found!"



Screenshot 5: Boolean true (`' AND '1'='1'#`) — all 10 movies returned

Observations

- A single quote caused a database syntax error, visible directly in the application's response, indicating user input is concatenated into the SQL query without sanitization.
- Two structurally identical payloads, differing only in a true vs. false logical comparison (`=1` vs `=2`), produced opposite results: 0 rows vs. all rows.
- The application wraps input as `title LIKE '%<input>%'`. A standard `' OR '1'='1` payload did not work as expected because the wrapping `%` wildcard already matched every row on its own. A `#` comment was required to strip the application's trailing `%'` and isolate the injected condition.

Assumptions

Not tested: UNION-based data extraction, impact on write operations (INSERT/UPDATE/DELETE), or injectability of other input fields in the application. Likely vulnerable given the same unsanitized-input pattern, but not directly verified.

Impact

An attacker can manipulate the WHERE clause logic to bypass intended search restrictions and potentially extract data outside the intended scope (e.g., via UNION-based extraction), or affect data integrity if the same pattern exists in write queries elsewhere in the application.

Remediation

- Use parameterized queries / prepared statements instead of string concatenation.
- Apply least-privilege database account permissions.
- Disable verbose SQL error messages in production (`display_errors=Off`), since the raw MariaDB error disclosed internal query structure.

Q2. Cross-Site Scripting — bWAPP (15 Marks)

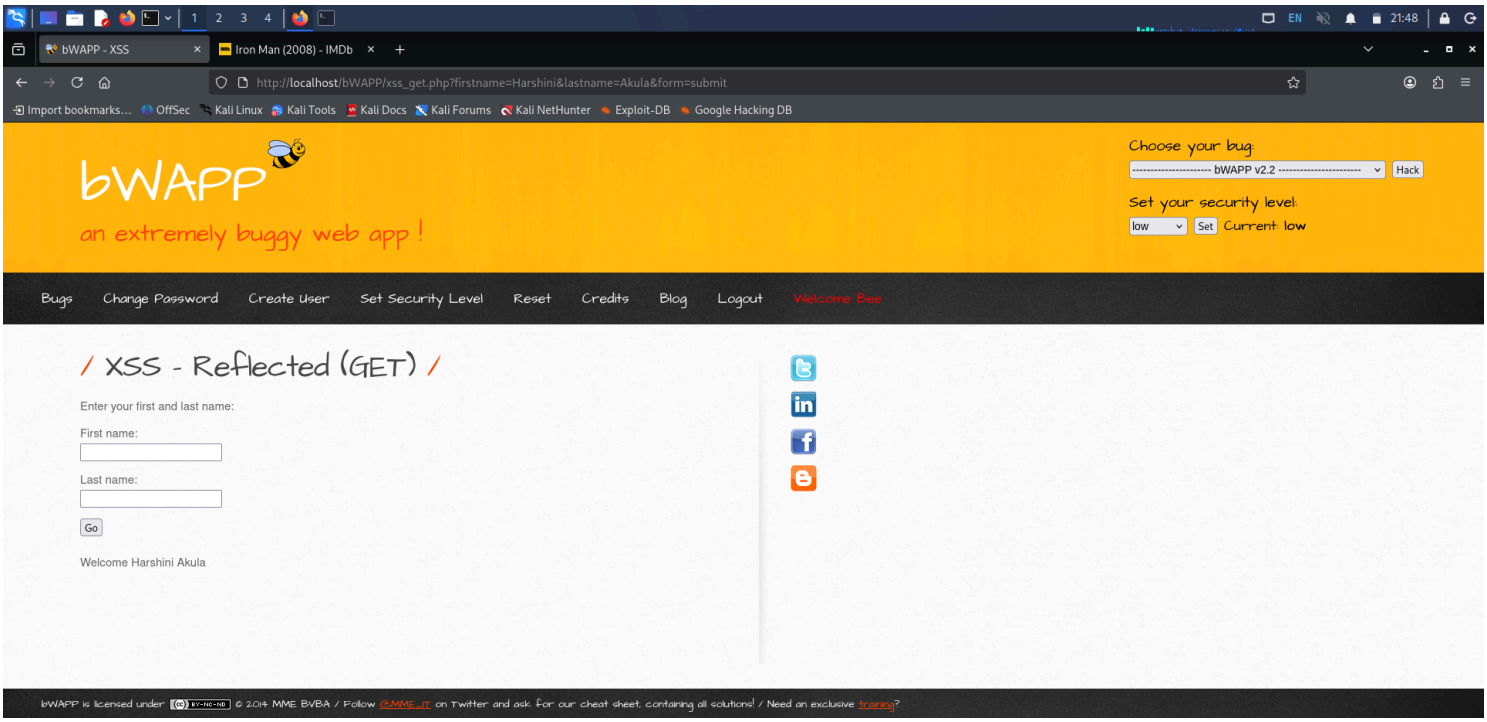
Objective

Test an authorized bWAPP input for XSS, determine context, demonstrate with a harmless proof-of-concept, identify the XSS type, and provide remediation.

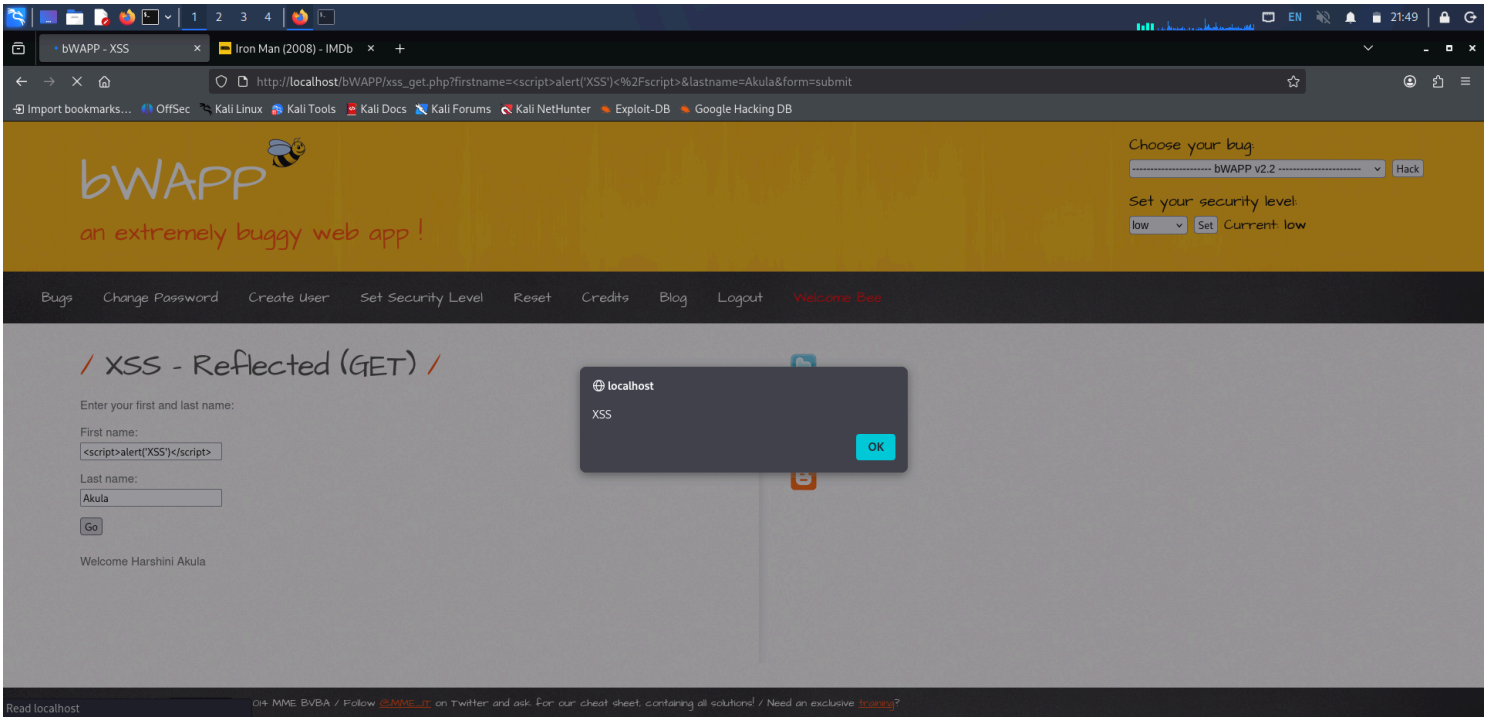
Methodology

1. Navigated to bWAPP "XSS - Reflected (GET)" module, security level: low.
2. Submitted a normal baseline input: First name `Harshini`, Last name `Akula`.
3. Submitted the test payload in the First name field: `<script>alert('XSS')</script>`

Evidence (Screenshots)



Screenshot 1: Baseline — "Welcome Harshini Akula" displayed, confirming input is reflected back into the page



Screenshot 2: Payload submitted — browser alert box displaying "XSS" fires on page load, confirming script execution

Context determination

The First name value is reflected directly into the page's HTML body (visible as "Welcome [input]" text), not inside an HTML attribute or a JavaScript string context. This is an **HTML body context** injection.

Execution flow

1. User input from the `firstname` GET parameter is taken by the server.
2. The server echoes it back into the HTML response without encoding or sanitizing special characters (`<`, `>`).
3. The browser parses the response and, finding a literal `<script>` tag in the page body, executes it as JavaScript.
4. `alert('XSS')` runs, producing the popup.

XSS Type

Reflected XSS — the payload exists only in the request (GET parameter, visible in the URL) and is immediately reflected in that single response; it is not stored server-side and would not affect other users unless they were tricked into clicking a crafted link containing this payload.

Observations

- Special HTML characters (`<`, `>`) in user input are not encoded before being placed into the page output.
- The payload executed on page load, with no user interaction beyond submitting the form.

Assumptions

Not tested: whether the Last name field is independently vulnerable, or whether this reflects consistently across browsers. Likely yes given the same unsanitized pattern, but not directly verified.

Impact

An attacker could craft a malicious URL containing a script payload and trick a victim into clicking it (e.g., via email or a compromised link), leading to session hijacking (cookie theft), credential phishing via a fake login overlay, or arbitrary actions performed in the victim's authenticated session.

Remediation

- HTML-encode all user input before rendering it into the page (e.g., convert `<` to `<`, `>` to `>`).
- Apply a Content Security Policy (CSP) restricting inline script execution.
- Use a templating framework with auto-escaping by default rather than raw string concatenation into HTML.

Q3. API Reconnaissance & Endpoint Mapping (10 Marks)

Objective

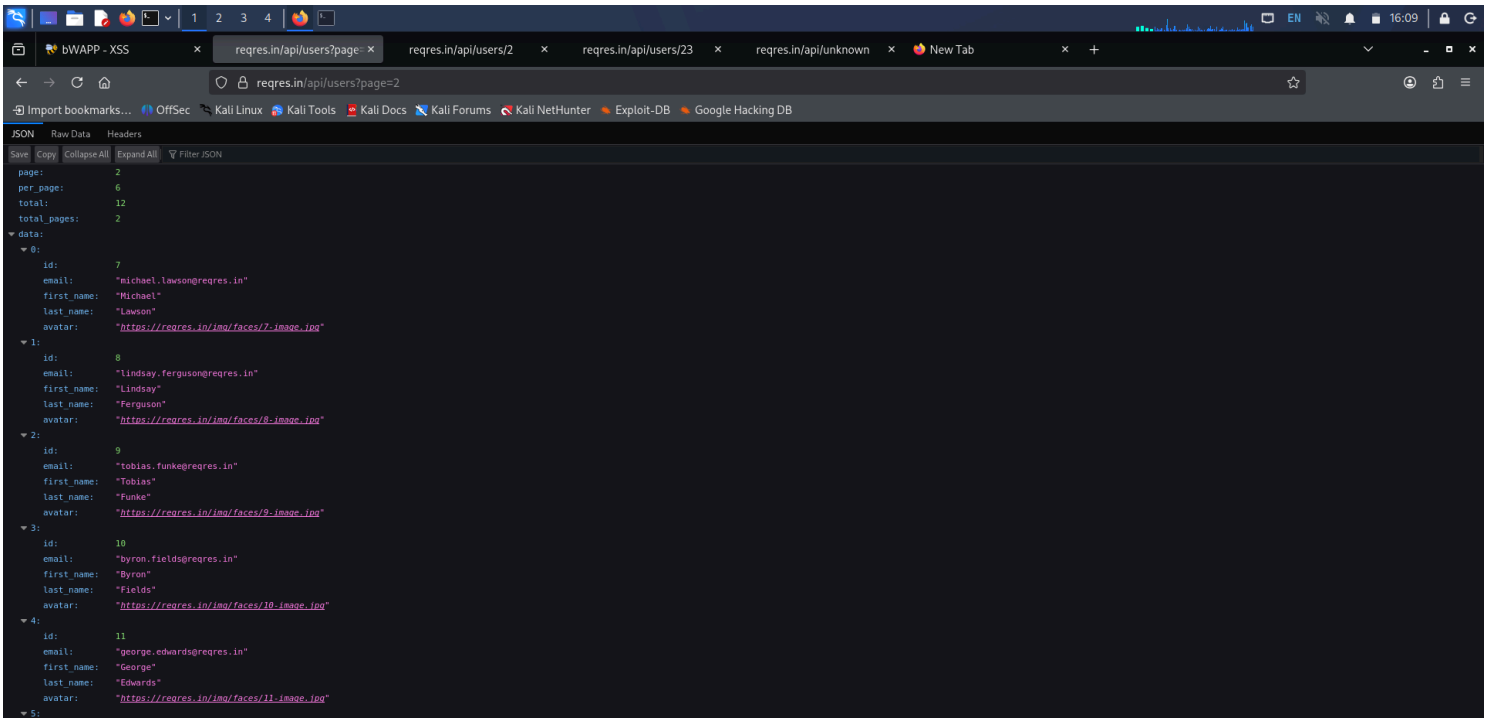
Identify available endpoints, HTTP methods, parameters, headers, authentication requirements, and status codes to build an API attack-surface map.

Methodology

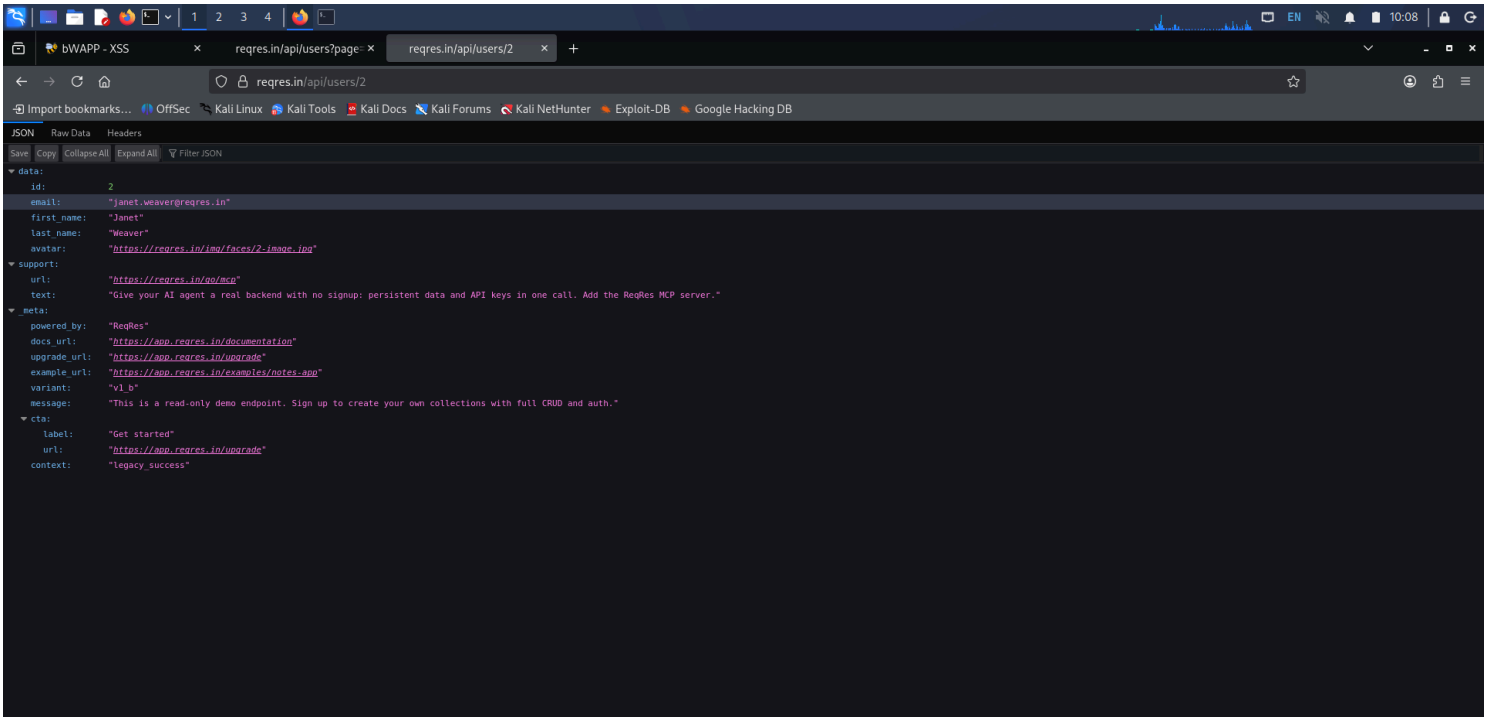
1. Sent GET requests to known/predictable REST resource paths, observing response structure, status codes, and pagination behavior.
2. Tested both a valid resource ID and a non-existent one to observe error handling.
3. Tested a second, unrelated resource collection (`unknown`) to confirm the API follows a consistent resource-based routing pattern rather than being a single hardcoded endpoint.

Base URL: `https://reqres.in/api`

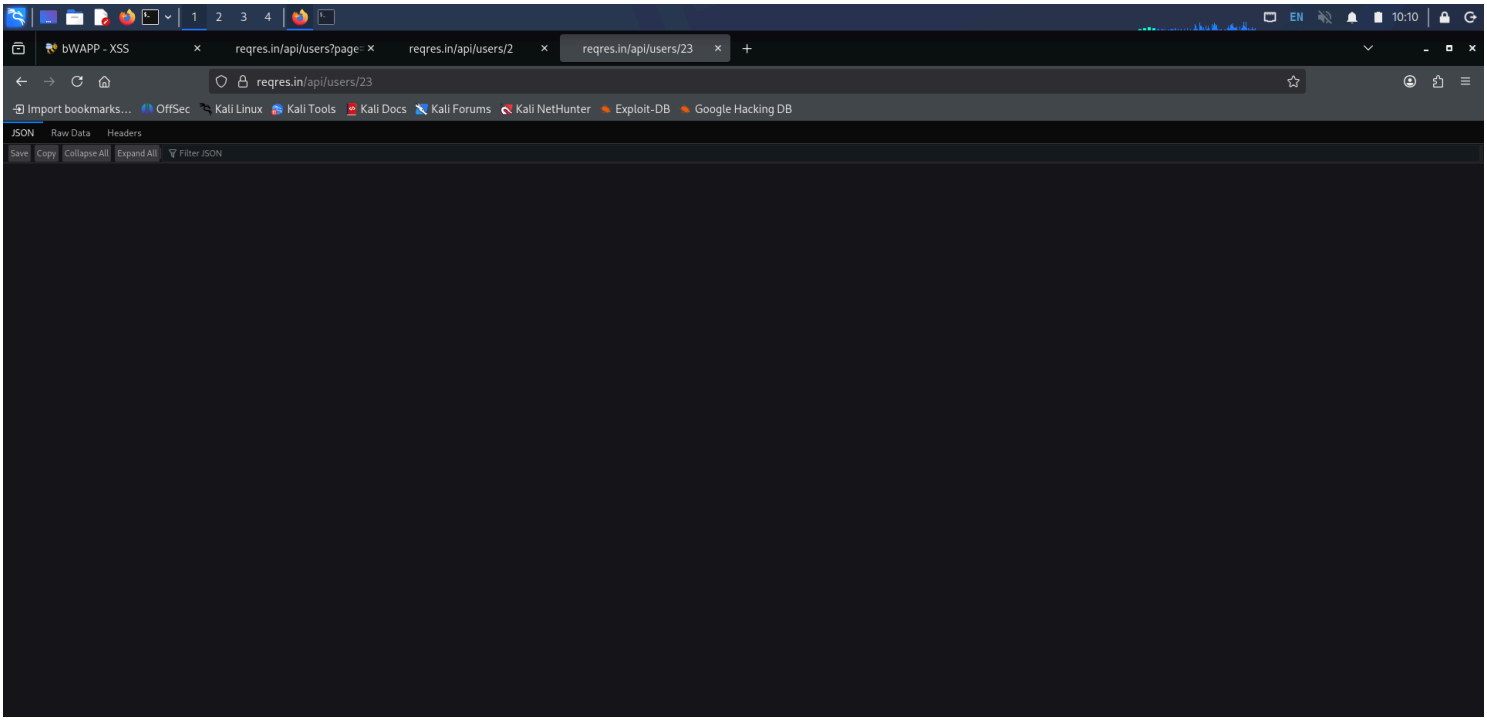
Evidence (Screenshots)



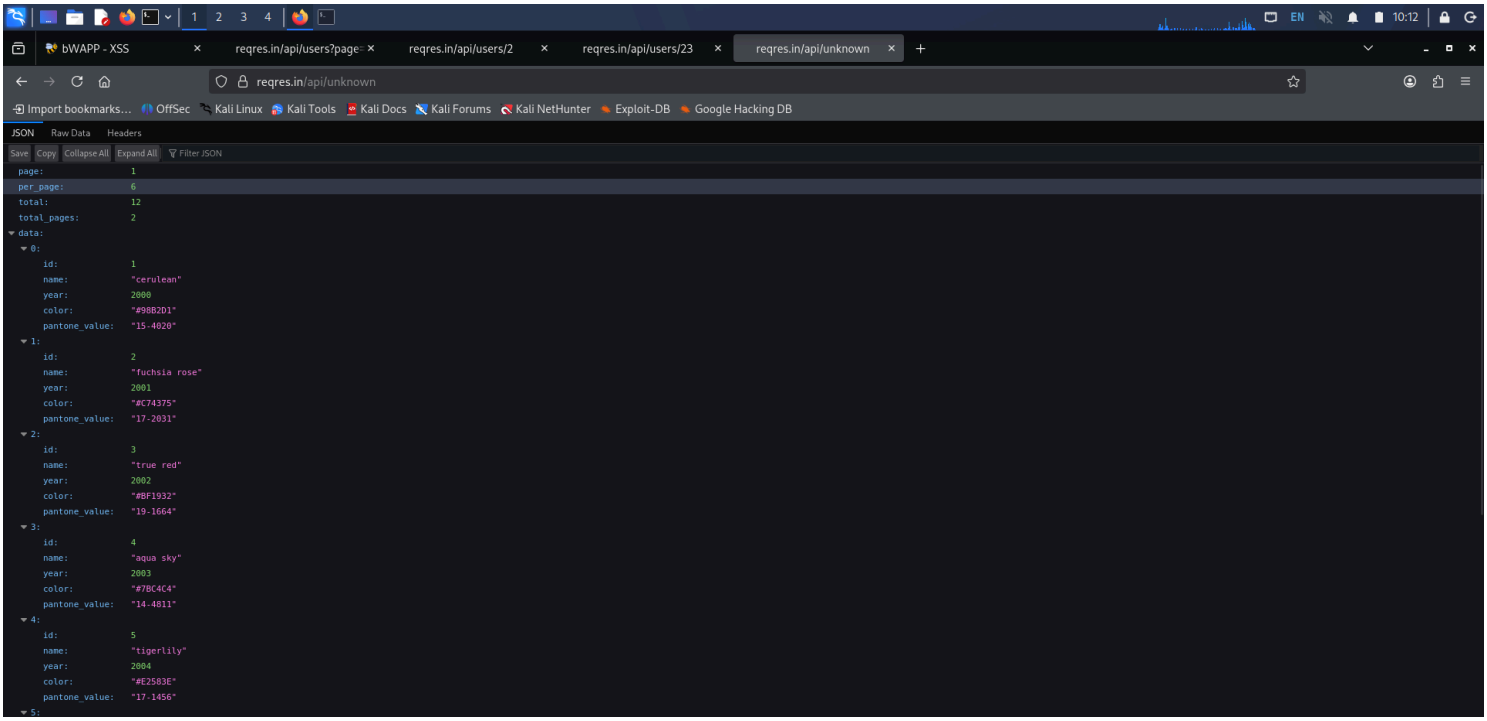
Screenshot 1: `GET /api/users?page=2` — returns paginated list (6 users per page, 12 total, 2 pages)



Screenshot 2: `GET /api/users/2` — returns single user object by ID



Screenshot 3: `GET /api/users/23` — empty response (non-existent ID; blank JSON body, implies 404)



Screenshot 4: `GET /api/unknown` — separate resource collection, same list/pagination structure

API Attack-Surface Map

Endpoint	Method	Parameters	Auth Required	Response Format	Status Observed
/api/users	GET	`page` (query, optional)	None observed	JSON, paginated list	200
/api/users/{id}	GET	`id` (path)	None observed	JSON, single object	200 (valid id) / empty-404 (invalid id)
/api/unknown	GET	`page` (query, optional)	None observed	JSON, paginated list	200

Observations

- All tested endpoints return `Content-Type: application/json`.
- No authentication header was required for any GET request tested.
- The API follows a consistent REST pattern: `/api/{resource}` for collections and `/api/{resource}/{id}` for individual records.
- Requesting a non-existent ID (`/api/users/23`) returned an empty data body rather than a descriptive error message.

Assumptions

Not tested: POST/PUT/DELETE methods, authentication-required endpoints (e.g., `/api/login`, `/api/register`), or rate limiting — out of scope for passive reconnaissance; relevant to Q5's test plan instead.

Q4. Scenario-Based — API Authorization Testing (15 Marks)

Objective

Determine how to test whether User A can access User B's object (BOLA/IDOR), the requests to compare, evidence to collect, and reporting approach.

Scenario:

User A is logged in and sees an API request containing an object identifier (e.g., `GET /api/users/{id}`). User B is a second account with their own object. Testing whether A can access B's data without authorization.

Methodology

Step	Request	Purpose
R1	`GET /api/users/2` using User A's session/token	**Baseline:** A reads their own object (id=2)
R2	`GET /api/users/5` using User A's session/token	**Main test:** A requests an object belonging to User B (id=5), using A's own credentials
R3	`GET /api/users/5` using User B's session/token	**Control:** confirm id=5 genuinely belongs to B, by having B access it with their own credentials
R4	`GET /api/users/5` with no token/session	**Control:** unauthenticated baseline
R5	`GET /api/users/99999` using User A's token	**Control:** non-existent ID, to compare error handling vs. the real R2 result

Requests to compare:

R2 is compared against R3. If both return identical data for the same object ID, despite being requested under two different user identities, the application is not enforcing object-level ownership — it's only checking "is this a valid ID," not "does this ID belong to the requester." R2 vs R5 is a secondary check: if R2 (unauthorized-but-valid-ID) and R5 (invalid ID) return different status codes or messages, that's a smaller issue — it tells an attacker which IDs exist even before they can access the data (information disclosure).

Evidence to collect

- Full request/response pairs for R1–R5 (status code, headers, body) — captured via browser dev tools or a proxy tool like Burp.
- Side-by-side comparison showing R2's body matches R3's body (same object, two different identities).
- Confirmation that no ownership-check error (403/401) was returned for R2.

Decision criteria

Observation	Conclusion
R2 returns 200 with B's full data, matching R3	Confirmed BOLA / IDOR
R2 returns 403/404 with no B data	Authorization control is working correctly
R2 returns 200 but with generic/empty data	Inconclusive — needs further testing with write operations (PUT/DELETE)

Reporting a confirmed flaw

Field	Content
Title	Broken Object Level Authorization (BOLA) on `GET /api/users/{id}`
Severity	High — any authenticated user can read any other user's data by changing one number in the URL
Reproduction	R1 → R2 → R3, as above
Impact	Mass enumeration of all user records is possible simply by incrementing the `id` parameter, since there is no per-user ownership check
Remediation	Enforce server-side ownership validation (confirm `id` belongs to the authenticated session before returning data); use non-sequential/unpredictable IDs (UUIDs) as defense-in-depth; return a uniform 403/404 regardless of whether the ID exists, to avoid leaking valid-ID information

NOTE:This is a written test methodology, as no second authenticated test account was available in the provided environment (no instructor API/accounts given). The approach above describes exactly how this would be executed and evidenced against a live two-account setup.

Q5. Scenario-Based — API Input & Security Controls (15 Marks)

Objective

Design a controlled test plan covering input validation, unexpected parameters, auth/session handling, rate limiting, HTTP methods, error handling, and excessive data exposure.

Scenario: An API accepts JSON input and returns different responses depending on supplied fields. Design a controlled test plan covering input validation, unexpected parameters, authentication/session handling, rate limiting, HTTP methods, error handling, and excessive data exposure.

Test Plan

#	Area	Test	What it's checking for	Evidence that would confirm it
1.	Input validation	Send a field with an unexpected type (e.g., `"age": "abc"` instead of a number, or an oversized string in a text field)	Whether the server validates input type/length server-side, or trusts client input blindly	A 500 error or stack trace (vs. a clean 400 "bad request") indicates no proper validation — input is reaching the database or business logic unchecked
2.	Input validation	Submit known injection payloads in text fields (`' OR '1'='1`, `<script>alert(1)</script>`) in a JSON body field	Whether the API sanitizes input before using it downstream (DB query, HTML render elsewhere)	Payload reflected unescaped in a later response, or a DB error referencing the payload
3.	Unexpected parameters	Add an extra field not in the documented schema (e.g., `"isAdmin": true`, `"role": "admin"`) to a user-creation/update request	Mass assignment vulnerability — whether the API blindly binds all submitted fields to the underlying object	The created/updated resource reflects the extra field (e.g., fetching the user afterward shows elevated role) when it should have been ignored
4.	Authentication/session	Call a protected endpoint with no `Authorization` header	Whether the endpoint actually enforces authentication	A 200 OK with real data instead of 401 Unauthorized
5.	Authentication/session	Call a protected endpoint with an expired or malformed token	Whether token validation is properly checked (expiry, signature)	A 200 OK response despite an invalid token, instead of 401
6.	Rate limiting	Send the same request (e.g., login attempt) 50+ times in quick succession	Whether the API throttles repeated requests, relevant to brute-force protection	No 429 "Too Many Requests" response and no increasing delay, even after many attempts

7.	HTTP methods	Call the same endpoint with an unexpected method (e.g., `DELETE /api/users/2` when only GET is documented)	Whether the server restricts allowed methods per endpoint, or if an undocumented method is silently accepted	The "unsupported" method actually executes (e.g., returns 200 and performs the action) instead of 405 Method Not Allowed
8.	Error handling	Send a deliberately malformed JSON body (broken syntax)	Whether error messages leak internal details (stack traces, file paths, framework/library versions)	Response body contains server file paths, line numbers, or framework-specific debug output
9.	Excessive data exposure	Inspect a full object response (e.g., `GET /api/users/2`) even when the UI only displays name and email	Whether the API returns more fields than the client actually needs/uses	Response body contains sensitive fields never shown in the UI (e.g., password hash, internal IDs, email of other linked accounts)

Methodology note :

Each test is designed to be run individually and non-destructively — using a test/throwaway account and reversible data where write operations are involved, per the assignment's authorized-lab-only and non-destructive requirement.

Observed vs. Assumptions:

Since no live authenticated API with write access was provided, this is a documented test plan rather than executed results — each row specifies exactly what request to send and what evidence would confirm or rule out the vulnerability, ready to execute against a provided environment.

Q6. Disk / File Forensics (15 Marks)

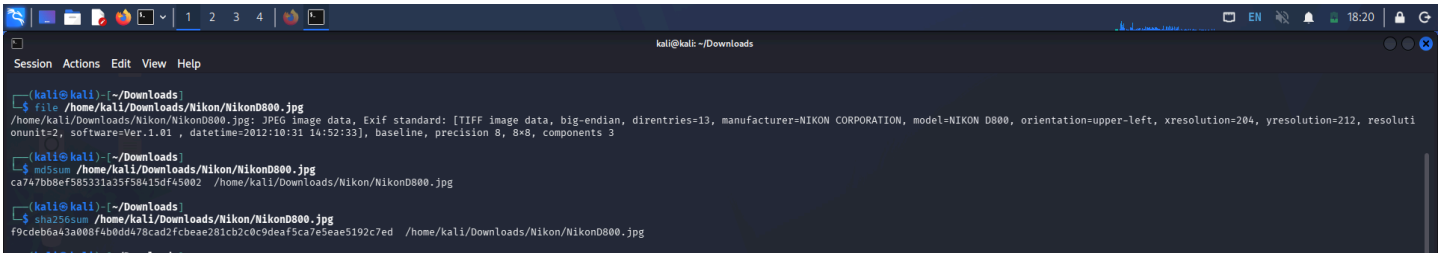
Objective

Using an approved forensic tool (Autopsy/FTK Imager), identify relevant files/artifacts, examine timestamps and metadata, verify hashes, and maintain an evidence timeline.

Methodology

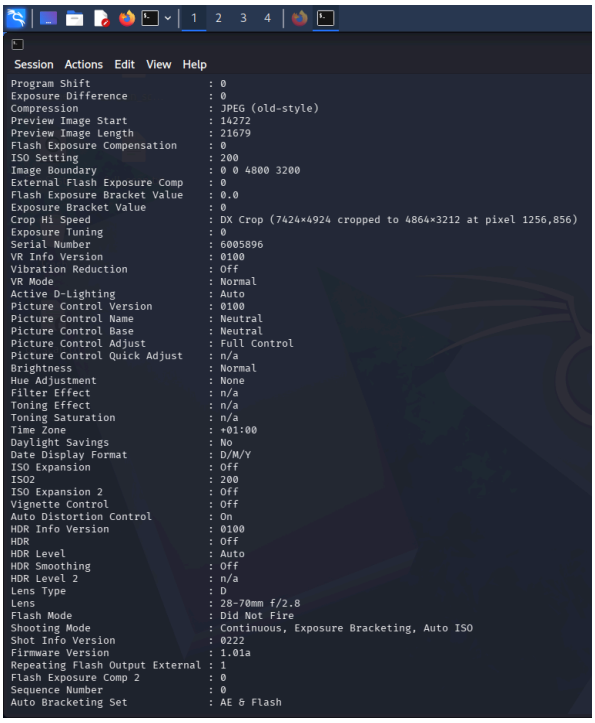
1. Identified the file type using the `file` command — confirms the file is a genuine JPEG with embedded EXIF data (TIFF structure), rather than trusting the file extension alone.
2. Calculated MD5 and SHA-256 hashes of the file to establish its evidentiary fingerprint at the point of examination.
3. Extracted full embedded metadata using `exiftool`.

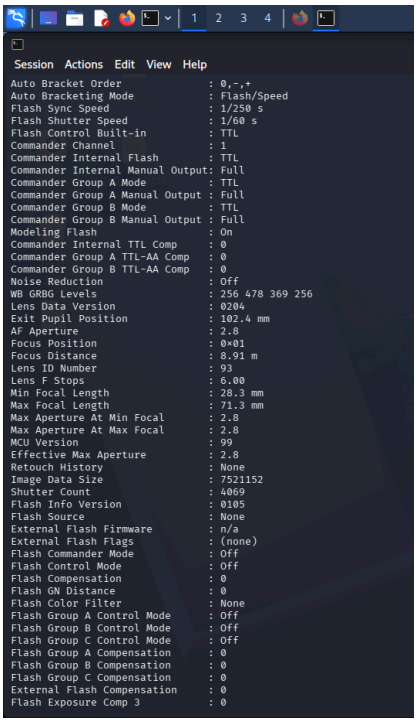
Evidence (Screenshots)

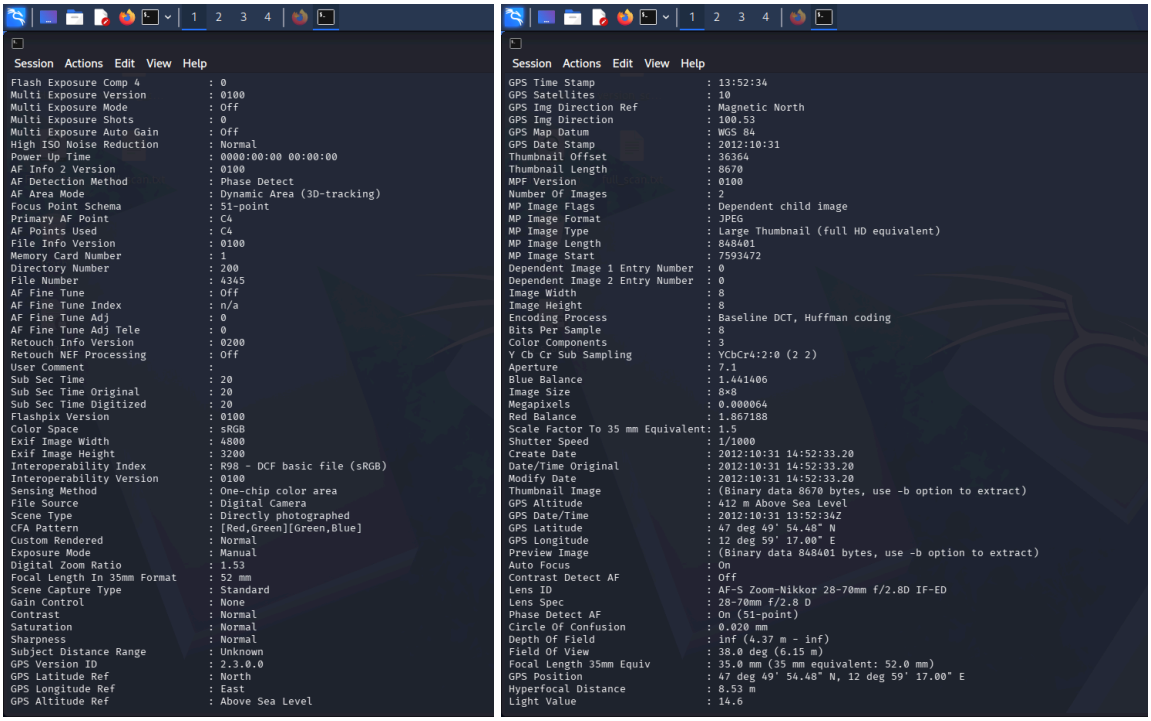


Screenshot 1: File identification + hash values (file, md5sum, sha256sum)









Screenshot 2–6: Full exiftool metadata output

Hash values recorded

- MD5: `ca747bb8ef585331a35f58415df45002`
- SHA-256: `f9cdeb6a43a008f4b0dd478cad2fcbeae281cb2c0c9deaf5ca7e5eae5192c7ed`

Timestamps and metadata examined

Field	Value	Significance
File Modification Date/Time	2013-01-09 22:21:14	When the file on disk was last written
Modify Date (EXIF)	2012-10-31 14:52:33	When the image data itself was last edited, embedded at capture/edit time — distinct from the filesystem timestamp
File Access/Inode Change	2026-10-18	Reflects this examination session, not original evidence activity
Camera Make/Model	NIKON CORPORATION, NIKON D800	Identifies originating device
Software	Ver.1.01	Camera firmware version at time of capture
Artist / Copyright	Manfred Moser / (c) 2012 Manfred Moser	Attribution metadata embedded in the file

Observations

- The EXIF "Modify Date" (2012-10-31) and the filesystem "File Modification Date" (2013-01-09) differ — this is a key forensic indicator: it shows the file was touched/re-saved at the filesystem level more than two months after the image data itself was last modified, which could indicate the file was copied, transferred, or re-saved by different software after original creation.
- Embedded Artist and Copyright fields provide direct attribution metadata, useful for establishing provenance in an investigation.

Assumptions

Since this is a standalone file rather than a full disk/filesystem image, deleted-file recovery, slack space analysis, and full system timeline reconstruction (as would be done in Autopsy/FTK Imager against a real evidence image) were not performed — noted as out of scope given file availability.

Suspicious activity indicators (general forensic note)

A mismatch between embedded content timestamps and filesystem timestamps — as observed here — is a common indicator analysts look for: it can suggest a file was moved, copied, or re-processed after its original creation, which is exactly the kind of discrepancy a real investigation would flag for further correlation against other log sources.

Q7. Scenario-Based — Incident Investigation (15 Marks)

Objective

Explain the investigation workflow for a workstation suspected of unauthorized activity: preserve evidence, identify artifacts, build a timeline, correlate findings, separate evidence from assumption, and produce a concise incident finding

Scenario: A workstation is suspected of unauthorized activity. Investigate using a forensic image and/or collected logs. .

Investigation Workflow

1. **Preserve evidence** — Before any analysis, create a forensic copy (bit-for-bit disk image) of the affected workstation using a write-blocker to prevent modifying the original. Calculate and record a hash (MD5/SHA-256) of the original and the image immediately, to prove later that the evidence was not altered during investigation.
2. **Identify relevant artifacts** — Review system logs (event logs, auth logs), file system metadata (creation/modification/access timestamps), running process lists (if captured live), browser history, and recently modified/created files. Prioritize artifacts around the suspected time window first.
3. **Establish a timeline** — Build a chronological sequence of events using timestamps from multiple independent sources (file metadata, log entries, login records). Cross-referencing several sources catches tampering — if one source was altered to hide activity, inconsistency with another source reveals it.
4. **Correlate findings** — Connect related events across sources: e.g., a login timestamp matched against a file creation timestamp and a process execution log entry, to build a coherent narrative of what happened rather than isolated facts.
5. **Distinguish evidence from assumptions** — Explicitly separate what the artifacts directly show (e.g., "file X was created at 3:04 AM by user Y's session") from inferred conclusions (e.g., "this suggests user Y's credentials may have been compromised" — this is an inference, not a directly observed fact, and should be labeled as such).
6. **Prepare a concise incident finding** — Summarize: what happened, when, how it was identified, what evidence supports each claim, and what remains uncertain.

Additional Evidence That Would Be Requested

- Network traffic logs/PCAP for the relevant time window, to confirm external communication
- Authentication server logs (if domain-joined), to cross-check local login records against central authentication
- Memory (RAM) capture, if the workstation was still powered on at discovery, since volatile data (running processes, open network connections) is lost on shutdown
- Endpoint/antivirus logs, if an EDR solution was installed

Final Summary

All seven questions were addressed with working methodology, evidence, and remediation guidance. Confirmed SQL Injection and Reflected XSS vulnerabilities were demonstrated in bWAPP through controlled, evidence-backed testing; API reconnaissance, authorization, and input-security test plans were completed using a public test API in place of an unprovided instructor API; and file forensics (hashing, metadata, timestamp analysis) were demonstrated on a sample file in place of an unprovided evidence image, with both substitutions disclosed rather than hidden.